



[BIT703 Web Technologies](#) / [Contents](#) / Topic 10: Server-side web application development with procedural and object-oriented database programming / Database operations in PHP

SQL injection protections 1m

Previously, we discussed SQL injection in PHP when introducing CRUD operations. When building web applications with databases, it's crucial to ensure that data from the client side is secure.

Directly using a value like `$ name = $_POST['name'];` is not considered best practice.

We don't know whether the variable 'name' exists, holds a valid value, is of reasonable length (not over 200 characters) or actually represents a person's name rather than random characters.

Assuming the checks above are in place, it's also recommended to use functions like **mysqli::escape_string** or **stripslashes** to handle special characters.

These methods help prevent SQL injection. In web apps, always validate client input – never assign `$ name = $_POST['name'];` without checks. Ensure that:

- ▶ the 'name' variable is set
- ▶ 'name' contains a suitable value
- ▶ the length of 'name' is within the desired length range
- ▶ 'name' has the right context - in this case, context is not as important as for the likes of an email address.

If the necessary checks and validations were not done, then something like the following could be executed.

[Skip to main content](#)

Given an example SQL for a login query below:

```
$query = "SELECT Email, Name, Password FROM Users WHERE Email = $email";
```

A hacker could pass something like the following in the login form for the email field (assuming no JavaScript is operating)

```
"100 or 1=1;"
```

Which creates the following SQL

```
SELECT Email, Name, Password FROM Users WHERE Email = 100 or 1=1;
```

This means the SQL will always work even if the email of 100 is wrong.

Always, always include data cleansing, validation and prepared SQL database functions in your PHP code.

For an example of SQL injection, refer to the article [SQL Injection in PHP: Practices to Avoid](#) (Marin, 2020) and review the code available in the GitHub repository.



Thoughts on this page?

